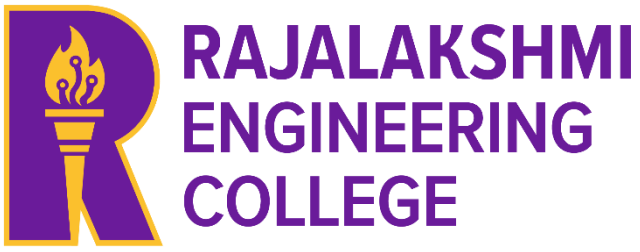
# RAJALAKSHMI ENGINEERING COLLEGE

**RAJALAKSHMI NAGAR, THANDALAM – 602 105**




## CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

**Laboratory Record Note Book**

| | | |
|---|---|---|
| **Name :** | : | PRAKASH A |
| **Year / Branch / Section** | : | II / BE Computer Science and Design / - |
| **University Register No.** | : | 2116251701041 |
| **College Roll No. :** | : | 251701041 |
| **Semester:** | : | III |
| **Academic Year :** | : | 2026 - 2027 |

# DYNAMIC PROGRAMMING

EXPERIMENT 16

# 1-DP-Playing with Numbers

*Dynamic Programming*

## CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

| | |
|---|---|
| **Started on** | Wednesday, 7 October 2026, 11:47 PM |
| **State** | Finished |
| **Completed on** | Saturday, 10 October 2026, 11:05 PM |
| **Time taken** | 2 days 23 hours |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00 Flag question

**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

**Example 1:**

***Input:*** *6*
***Output:****6*
***Explanation:*** *There are 6 ways to 6 represent number with 1 and 3*

*1+1+1+1+1+1*
*3+3*
*1+1+1+3*
*1+1+3+1*
*1+3+1+1*
*3+1+1+1*

**Input Format**

First Line contains the number n

**Output Format**

**Print: The number of possible ways 'n' can be represented using 1 and 3**

Sample Input

6

Sample Output

6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n, i;
    scanf("%d", &n);

    long long dp[n + 1];
    dp[0] = 1;

    for (i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];
        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 6 | 6 | 6 | ✔ |
| ✔ | 25 | 8641 | 8641 | ✔ |
| ✔ | 100 | 24382819596721629 | 24382819596721629 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

EXPERIMENT 17

# 2-DP-Playing with chessboard

*Dynamic Programming*

## CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

| | |
|---|---|
| **Started on** | Saturday, 10 October 2026, 11:09 PM |
| **State** | Finished |
| **Completed on** | Saturday, 10 October 2026, 11:11 PM |
| **Time taken** | 2 mins 20 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00 Flag question

**Playing with Chessboard:**

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:**

**Input**

3

**1** 2 4

**2** 3 4

**8 7 1**

**Output:**

19

**Explanation:**

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

**Input Format**

First Line contains the integer n

The next n lines contain the n*n chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n, a[100][100], i, j;
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);

        }

    }

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(i > 0 && j > 0)
            {
                if(a[i-1][j] > a[i][j-1])
                    a[i][j] += a[i-1][j];
                else
                    a[i][j] += a[i][j-1];
            }
            else if(i > 0)
                a[i][j] += a[i-1][j];
            else if(j > 0)
                a[i][j] += a[i][j-1];
        }
    }

    printf("%d", a[n-1][n-1]);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 2 4<br>2 3 4<br>8 7 1 | 19 | 19 | ✔ |
| ✔ | 3<br>1 3 1<br>1 5 1<br>4 2 1 | 12 | 12 | ✔ |
| ✔ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28 | 28 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

EXPERIMENT 18

# 3-DP-Longest Common Subsequence

*Dynamic Programming*

## CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

| | |
|---|---|
| **Started on** | Saturday, 10 October 2026, 11:13 PM |
| **State** | Finished |
| **Completed on** | Saturday, 10 October 2026, 11:16 PM |
| **Time taken** | 2 mins 53 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** Correct Mark 1.00 out of 1.00 Flag question

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| s1 | a | g | **g** | **t** | **a** | **b** | |
| s2 | **g** | x | **t** | x | **a** | y | **b** |

**The length is 4**

Solveing it using Dynamic Programming

**For example:**

| Input | Result |
|---|---|
| aab<br>azb | 2 |

**Answer:** (penalty regime: 0 %)

```
#include <stdio.h>
#include <string.h>

int main() {
    char a[100], b[100];
    int dp[100][100] = {0}, i, j;

    scanf("%s %s", a, b);

    for(i = 1; i <= strlen(a); i++)
    {
        for(j = 1; j <= strlen(b); j++)
        {
            if(a[i-1] == b[j-1])
            {

                dp[i][j] = dp[i-1][j-1] + 1;
            }
            else if(dp[i-1][j] > dp[i][j-1])
            {
                dp[i][j] = dp[i-1][j];
            }

            else
            {
                dp[i][j] = dp[i][j-1];
            }

        }
    }

    printf("%d", dp[strlen(a)][strlen(b)]);

    return 0;
}

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | aab<br>azb | 2 | 2 | ✔ |
| ✔ | ABCD<br>ABCD | 4 | 4 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

EXPERIMENT 19

# 4-DP-Longest non-decreasing Subsequence

*Dynamic Programming*

## CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

| | |
|---|---|
| **Started on** | Saturday, 10 October 2026, 11:18 PM |
| **State** | Finished |
| **Completed on** | Saturday, 10 October 2026, 11:19 PM |
| **Time taken** | 41 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** Correct Mark 1.00 out of 1.00 Flag question

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int n, a[100], dp[100], i, j, max = 0;

    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }

    for(i = 1; i < n; i++)
    {
        for(j = 0; j < i; j++)
        {
            if(a[j] <= a[i])
            {
                if(dp[j] + 1 > dp[i])
                {
                    dp[i] = dp[j] + 1;
                }
            }
        }
    }

    for(i = 0; i < n; i++)
    {
        if(dp[i] > max)
        {
            max = dp[i];
        }
    }

    printf("%d", max);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9<br>-1 3 4 5 2 2 2 2 3 | 6 | 6 | ✔ |
| ✔ | 7<br>1 2 2 4 5 7 6 | 6 | 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.